1. Interrupciones, sincronización y *callbacks*

Cuando salta una interrupción (cuya dirección está almacenada en la tabla de vectores), el hardware del procesador suspende la ejecución del hilo principal (`main`), guarda automáticamente el contexto (registros `R0-R3`, `R12`, `LR`, `PC`, `xPSR`) en la pila y salta a la ISR asignada en la tabla de vectores.

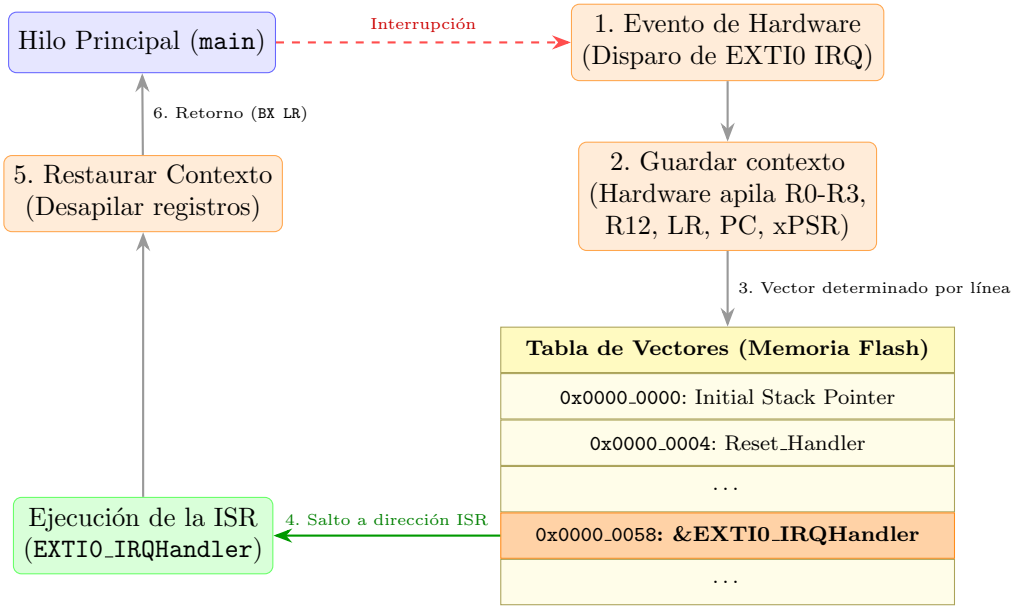


Figura 1: Mecanismo de atención a interrupciones y lectura de puntero en la Tabla de Vectores.

Al compartir variables entre el hilo principal y una rutina de interrupción, surgen problemas de inconsistencia de datos y condiciones de carrera:

- **Uso obligatorio de `volatile`:** Evita que el compilador almacene en registros de la CPU el valor de una variable que puede ser alterada por la ISR en cualquier momento.
- **Operaciones no atómicas:** Si el hilo principal modifica una variable de 32 bits mientras ocurre una interrupción a mitad del proceso de escritura, la ISR leerá datos corruptos. Normalmente, las variables de 32 bits ya se gestionan de manera atómica automáticamente en ARM, pero es mejor protegerlas por si las moscas en caso de poder existir condiciones de carrera.

```c
#include <stdint.h>

volatile uint32_t contador_eventos = 0;

void TIM2_IRQHandler(void) {
    // esta función se ejecuta automáticamente cuando hay interrupción
    contador_eventos++;
}

uint32_t leer_contador_seguro(void) {
    uint32_t copia;

    // Desactivar interrupciones antes de leer
    __asm volatile ("cpsid i" : : : "memory");
    copia = contador_eventos; // Lectura protegida
```

```
    // Reactiva interrupciones tras leer
    __asm volatile ("cpsie i" : : : "memory");

    return copia;
}
```

1.1. Barreras de memoria

El código `C` mantiene un orden lógico. Sin embargo, las optimizaciones del compilador, y el reordenamiento dinámico de instrucciones que sucede a nivel arquitectural, pueden cambiar dicho orden, a fin de mejorar el rendimiento. Esto siempre se hace de tal manera que el resultado final es el mismo, siempre y cuando no haya modificaciones externas de los valores implicados. ¿Y qué pasa con un sistema empotrado con periféricos mapeados en memoria? Pues que puede haber modificaciones externas.

En ciertos momentos, querremos obligar al procesador a completar las operaciones de memoria antes de continuar. Para ello, la arquitectura ARM Cortex-M provee tres instrucciones de barrera de memoria:

- `__DMB()` **(Data Memory Barrier):** Garantiza que todas las accesos a memoria (lecturas y escrituras) previos a la barrera se hayan completado antes de que se inicie cualquier acceso a memoria posterior. No frena la ejecución de instrucciones que no accedan a memoria.

```
__attribute__((always_inline)) static inline void __DSB(void) {
    __asm volatile ("dmb 0xF":::"memory");
}
```

- `__DSB()` **(Data Synchronization Barrier):** Detiene la ejecución de **todas** las instrucciones hasta que las operaciones de acceso a memoria pendientes hayan finalizado.

```
__attribute__((always_inline)) static inline void __DMB(void) {
    __asm volatile ("dsb 0xF":::"memory");
}
```

- `__ISB()` **(Instruction Synchronization Barrier):** Completa todas las instrucciones pendientes del procesador y fuerza a volver a cargar las instrucciones desde memoria o caché. Se utiliza tras modificar la configuración del sistema (como el mapa de memoria o la MPU) para evitar mantener valores antiguos.

```
__attribute__((always_inline)) static inline void __ISB(void) {
    __asm volatile ("isb 0xF":::"memory");
}
```

Generalmente, los manuales de uso nos indicarán cuándo es necesario utilizar una de estas barreras al interactuar con los diferentes periféricos y elementos del procesador.

Un caso de uso de `__DSB()` en microcontroladores ocurre al limpiar bits de interrupción en una ISR. Debido al tiempo que tarda el bus en procesar la escritura en el periférico (potencialmente miles de ciclos si el reloj del periférico es muy lento), la CPU podría salir de la ISR antes de que la bandera se limpie físicamente en el registro, re-disparando la misma interrupción.

```c
void EXTI0_IRQHandler(void) {
    // ... gestión de ISR

    // ... limpieza de bits
    EXTI->PR = EXTI_PR_PR0;
    // ... garantizamos que se haya escrito antes de salir
    __DSB();
}
```

1.2. Punteros a función y Callbacks

Para mantener un diseño de software modular, la capa que interactúa directamente con el hardware (*drivers*) no debe contener lógica de aplicación. El mecanismo estándar para notificar eventos desde dichos *drivers* (o una ISR) hacia la aplicación es la utilización de **punteros a función**. Esta técnica se conoce comúnmente como *callbacks*, ya que la función *hardware* hará una llamada (*callback*) a nuestra función con lógica.

Un puntero a función almacena la dirección de memoria donde está situada, permitiendo su invocación dinámica en tiempo de ejecución. Podemos ver un esquema en la Figura 2.

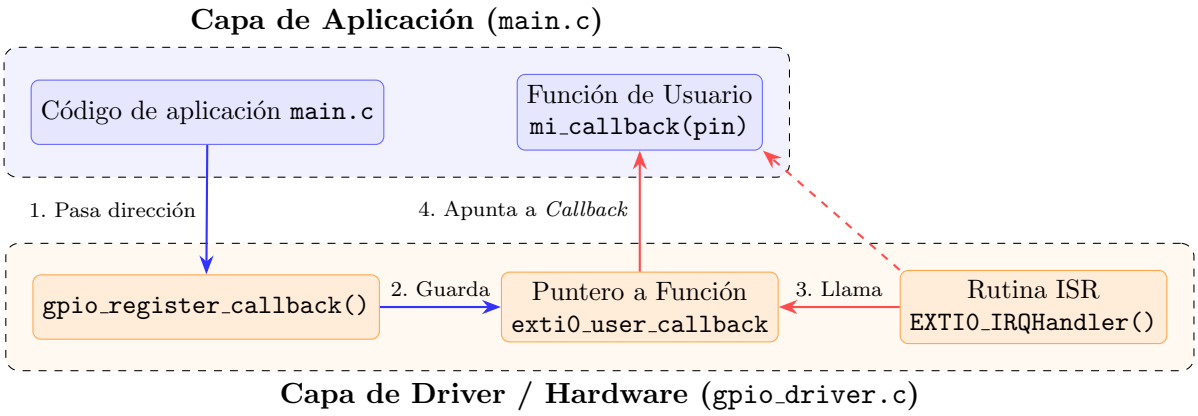


Figura 2: Mecanismo de Registro y Ejecución de un Call-back.

Hay que tener en cuenta que los *callback* se ejecutan desde la interrupción, por tanto el procesador está en modo de atención de ISRs. Estos callbacks no deben incluir procesamiento pesado, y únicamente marcar los valores necesarios o tareas pendientes para que el hilo principal, al volver, los gestione. De lo contrario, estaríamos retrasando cualquier otra llamada a interrupción durante la duración de ejecución del *callback*.

```c
#include <stddef.h>
#include <stdint.h>

// Definición del tipo de puntero a función
// Será una función que no devuelve nada (void)
// Y que recibe un argumento tipo uint16_t
typedef void (*gpio_callback_t)(uint16_t pin);

// Variable del driver para almacenar el callback
static volatile gpio_callback_t exti0_user_callback = NULL;

// Función de la API pública para registrar el callback
// desde el código de aplicación (p.ej: main.c)
```

```
14 void gpio_register_callback(gpio_callback_t callback) {
15     exti0_user_callback = callback;
16 }
17
18 // Manejador de la interrupción del hardware
19 void EXTI0_IRQHandler(void) {
20     // ... procesamiento de ISR
21     uint16_t pin = ...;
22
23     // limpieza de bits de interrupción
24     EXTI->PR = EXTI_PR_PR0;
25     __DSB();
26
27     // Llamada al callback
28     if (exti0_user_callback != NULL) {
29         exti0_user_callback(pin);
30         // Ejecuta la función de la capa de aplicación
31         // avisando del estado (en este caso el pin)
32     }
33 }
```